

Structuring and Designing Code

Across Software Architecture Styles

A practical guide to Layered, Modular, Monolithic, Microservice, Peer-to-Peer, and Client-Server architectures: how to organize folders, draw boundaries, and make design decisions that hold up as a project grows.

Prepared for: Induwara

How to Use This Guide

These six terms get used loosely, and they are not all the same kind of thing. Layered, modular, and monolithic describe how code is organized **inside one deployable unit**. Microservice, peer-to-peer, and client-server describe how **separate running processes** talk to each other over a network. A real system usually combines one from each group, for example a modular monolith exposed through a client-server API, or a set of microservices where each service is internally layered.

Each section below covers the core idea, how to structure folders and code, the design principles that matter most, common pitfalls, and when to reach for it.

1. Layered Architecture

Code is organized into horizontal layers, each with one responsibility. A layer only talks to the layer directly below it. The classic stack is Presentation, Business Logic, and Data Access, sometimes with a Domain layer added underneath.

Structure

```
src/
  presentation/  -> controllers, views, API routes
  service/       -> business logic, use cases
  repository/    -> database access, queries
  domain/        -> entities, value objects
  config/        -> app wiring, dependency setup
```

Design principles

- **One-way dependency rule.** Presentation depends on Service, Service depends on Repository. Never let a lower layer import from a higher one.
- **Interfaces at the boundary.** Define a repository interface in the service layer and implement it in the data layer, so business logic never imports a specific database driver directly.
- **Keep layers thin where possible.** Controllers should only parse input and call a service method; they should not contain business rules.
- **Do not skip layers.** A controller calling the database directly defeats the point of having layers at all, even if it feels faster to write.
- **Data transfer objects (DTOs) at each boundary** keep internal domain models from leaking into the API response shape.

Common pitfalls

- "Anemic" layers where business logic slowly migrates into controllers because it's convenient, leaving the service layer as a thin, pointless pass-through.
- Turning it into a rigid maze for a small app: three layers for a 200-line script adds ceremony without benefit.

When to use it

Good default for most CRUD-heavy applications, university projects, and APIs where the team is small and the domain logic is moderate. It is easy to explain and easy to test layer by layer.

Interfaces and design patterns

- **Repository pattern.** Define a `UserRepository` interface in the service or domain layer with methods like `find_by_id`, and implement it in the data layer. The service layer depends only on the interface, never the concrete SQL or ORM class.
- **Dependency Injection.** Pass repository and service instances into the classes that need them (through a constructor or a DI container) instead of constructing them inline. This is what makes swapping a real database for a fake one in tests possible.
- **DTO / Mapper pattern.** Use small mapper functions or classes to convert between domain entities and the DTOs returned by the presentation layer, so a database schema change does not automatically change the public API shape.
- **Strategy pattern** inside the service layer when the same operation needs different logic depending on context, for example different pricing rules per customer type, without branching logic scattered across the codebase.
- **Interface segregation.** Keep repository and service interfaces narrow and specific to what each caller actually needs, rather than one large interface every class must implement in full.

```
# service layer defines the contract
class UserRepository(Protocol):
    def find_by_id(self, user_id: int) -> User: ...
    def save(self, user: User) -> None: ...

# data layer provides the implementation
class SqlUserRepository:
    def find_by_id(self, user_id: int) -> User: ...
    def save(self, user: User) -> None: ...
```

2. Modular Architecture

Code is organized by **feature or domain** rather than by technical role. Each module is a self-contained slice of the system, for example "orders", "users", or "inventory", and each module can internally be layered.

Structure

```
src/
  modules/
    users/
      controller.py
      service.py
      repository.py
      models.py
```

```
orders/  
  controller.py  
  service.py  
  repository.py  
  models.py  
inventory/  
  ...  
shared/  
  utils/  
  config/
```

Design principles

- **High cohesion inside a module, low coupling between modules.** Everything about "orders" lives in the orders folder.
- **Modules communicate through explicit interfaces**, not by importing each other's internal files directly. A public `api.py` or `index` file per module is a common pattern.
- **Shared code goes in a clearly separate shared/common folder**, and should stay small; if "shared" keeps growing, some of it is probably a module in disguise.
- **Each module should be deletable.** A good test of modularity is: could you remove this folder and only break the things that explicitly depend on it?
- **Avoid circular dependencies between modules**; if orders needs users and users needs orders, that is a sign the boundary is drawn in the wrong place.

Common pitfalls

- Modules that quietly reach into each other's internal files instead of going through the public interface, which erodes the boundary over time.
- Splitting too early into many tiny modules before the domain is well understood, which creates more coordination overhead than it saves.

When to use it

Best for applications with several distinct domains that are expected to grow, or for a monolith that might later be split into microservices. Modular structure inside a single deployable is often called a **modular monolith**, and it is a strong middle ground between a tangled monolith and full microservices.

Interfaces and design patterns

- **Facade pattern.** Each module exposes one public entry point (a class or a small set of functions in an `api.py` / `index.ts`) that hides its internal files. Other modules only ever import from that facade.
- **Observer / event bus pattern** for cross-module communication that shouldn't create a direct dependency, for example the orders module publishing an "OrderPlaced" event that the notifications module subscribes to, without either module importing the other directly.

- **Dependency inversion between modules.** If module A needs something from module B, define the interface in A and let B implement it, rather than A depending on B's concrete classes directly.
 - **Factory pattern** for wiring modules together at startup, keeping the "who depends on what" decisions in one composition-root file instead of scattered across modules.
 - **Adapter pattern** when one module needs data shaped differently than another module provides it, translating at the boundary instead of leaking one module's internal shape into another.
-

3. Monolithic Architecture

The entire application, UI, business logic, and data access, ships and runs as a single deployable unit. Monolithic is a statement about **deployment**, not about internal organization; a monolith can and should still be layered or modular inside.

Structure

```
app/  
  api/           -> all HTTP routes  
  services/      -> business logic  
  models/        -> ORM models / entities  
  db/            -> migrations, connection setup  
  static/        -> frontend assets, if served together  
  main.py        -> single entry point, one process
```

Design principles

- **Impose internal boundaries deliberately.** Use layered or modular organization inside the monolith so it does not degenerate into a single giant file over time.
- **Single database, but logically partitioned schemas or tables per domain** makes a future split much easier if one is ever needed.
- **One build, one deploy, one process**, which simplifies transactions, debugging, and local development compared to a distributed system.
- **Keep a clear entry point** and avoid global mutable state scattered across the codebase; it becomes very hard to reason about as the app grows.
- **Automated tests matter more here**, since a single bad change can bring down the whole application rather than just one isolated service.

Common pitfalls

- The "big ball of mud": no internal structure, everything imports everything, and no one can safely change one part without fear of breaking another.
- Scaling the whole application to handle load in one small part of it, because monoliths scale as a unit rather than piece by piece.

When to use it

The right default for most new projects, student projects, startups, and small teams. It is simpler to build, test, deploy, and reason about than a distributed system, and premature microservices are a well-known source of unnecessary complexity.

Interfaces and design patterns

- **Composition root / Factory pattern.** One place (often `main.py` or a startup module) constructs and wires every service, repository, and controller, so dependencies are explicit rather than built ad-hoc throughout the codebase.
 - **Plugin / Strategy pattern** for optional or swappable behavior, for example different payment providers implementing a shared `PaymentGateway` interface, so new providers can be added without touching existing code.
 - **Unit of Work pattern** to group several repository operations into a single transaction, which is straightforward in a monolith since everything shares one database connection.
 - **Internal module interfaces** (borrowed from the modular pattern above) even inside a monolith, so that if it ever needs to be split into services later, the seams already exist.
 - **Domain events** raised internally (in memory, not over a network) to decouple side effects, for example sending a welcome email after user registration, from the core registration logic.
-

4. Microservice Architecture

The system is split into small, independently deployable services, each owning its own data and business logic for one bounded capability. Services communicate over the network, typically via REST, gRPC, or a message queue.

Structure

```
services/  
  user-service/  
    src/ (its own layered or modular structure)  
    Dockerfile  
    db/ (its own database, not shared)  
  order-service/  
    src/  
    Dockerfile  
    db/  
  notification-service/  
    src/  
    Dockerfile  
gateway/      -> API gateway, routes external requests  
infra/        -> service discovery, message broker config
```

Design principles

- **Each service owns its own data.** No service reaches directly into another service's database; all access goes through that service's API.

- **Bounded contexts, not technical layers, define service boundaries.** "Billing" and "Shipping" are good boundaries; "the database service" is not.
- **Design for failure.** Use timeouts, retries, and circuit breakers, since any service can be slow or unavailable independently of the others.
- **Prefer asynchronous messaging for cross-service workflows** where strict consistency is not required, to avoid tight coupling and cascading failures.
- **Version APIs explicitly** so one service can evolve without breaking every service that calls it.
- **Centralize logging, tracing, and monitoring** from day one, since debugging across many processes without it is very difficult.

Common pitfalls

- Splitting a system into microservices before the domain boundaries are well understood, which usually just distributes the mess instead of fixing it.
- A "distributed monolith": many services that must all be deployed together because they are too tightly coupled, giving you the operational cost of microservices with none of the independence benefit.
- Chatty communication, where one user action triggers a long chain of synchronous calls across many services, hurting both latency and reliability.

When to use it

Makes sense once a system and team have grown large enough that independent deployment, independent scaling, or independent ownership by different teams is genuinely needed. For small projects and solo work, the operational overhead usually outweighs the benefit.

Interfaces and design patterns

- **API Gateway pattern.** A single entry point routes external requests to the right service, handles auth, and can aggregate responses from multiple services so clients don't need to know the internal topology.
- **Circuit breaker pattern.** Wrap calls to other services so that repeated failures "trip" the breaker and fail fast instead of piling up slow, doomed requests and taking down the caller too.
- **Adapter pattern** for every external service call, isolating the specific protocol (REST, gRPC, a message format) behind a clean interface so the calling service's core logic doesn't depend on transport details.
- **Saga pattern** for transactions that span multiple services, coordinating a sequence of local transactions with compensating actions instead of a single distributed transaction.
- **CQRS (Command Query Responsibility Segregation)** when read and write patterns differ a lot, separating the model used to update data from the model used to query it, sometimes backed by different stores.
- **Contract-first interfaces.** Define each service's API with an explicit schema (OpenAPI, gRPC .proto files, or an event schema for a message queue) before writing the implementation, so client and server teams can work in parallel against the same contract.

5. Peer-to-Peer (P2P) Architecture

There is no central server. Every node, or peer, can act as both client and server, requesting resources from other peers and providing resources to them. Used in file sharing, blockchain networks, and some real-time collaboration tools.

Structure

```
node/  
  network/  
    peer_discovery.py  -> finding other peers  
    connection.py      -> handling inbound/outbound links  
    protocol.py        -> message format, handshake rules  
  storage/  
    local_store.py     -> what this peer holds/caches  
  sync/  
    replication.py     -> keeping shared state consistent  
  main.py              -> starts node, joins the network
```

Design principles

- **Design the wire protocol first.** Peers only agree if their message format, handshake, and versioning rules are precisely defined and consistent across all nodes.
- **Plan for peer discovery.** New nodes need a way to find existing peers, whether through a bootstrap list, a DHT (distributed hash table), or a rendezvous point.
- **Assume peers are unreliable.** They join, leave, and fail constantly, so the protocol must tolerate missing peers without breaking the whole network.
- **Decide on a consistency model up front:** eventual consistency is far more common and far easier to build than strong consistency across independent peers.
- **Keep node logic separable from network logic,** so the core behavior (what a peer does with data) can be tested without spinning up a real network.

Common pitfalls

- Underestimating churn: treating the network as if peers are always present leads to code that breaks constantly in the real, unreliable network.
- No conflict resolution strategy for when two peers hold different versions of the same data at once.
- Security: without central authority, verifying that a peer is who it claims to be, and that its data has not been tampered with, needs to be designed in from the start.

When to use it

Fits systems that need to work without a central point of failure or ownership: file sharing, decentralized applications, blockchain, and some multiplayer or collaboration tools. Rarely the right choice for a typical web or mobile application.

Interfaces and design patterns

- **Protocol interface abstraction.** Define a `PeerProtocol` interface (message types, serialization, handshake steps) separately from the transport (TCP, UDP, WebRTC), so the transport can change without touching protocol logic.
 - **Observer pattern** for propagating events through the network: a node observes incoming messages and notifies interested local components (storage, sync, UI) without those components needing to know about networking directly.
 - **Strategy pattern** for peer roles or routing algorithms, for example switching between flooding, gossip, or DHT-based routing behind a common `RoutingStrategy` interface.
 - **State pattern** for a peer's connection lifecycle (discovering, handshaking, connected, disconnected), keeping transition logic explicit instead of scattered boolean flags.
 - **Command pattern** for queued or replayable operations, useful when a peer needs to retry sending data after reconnecting, or replay a message log during sync.
-

6. Client-Server Architecture

A central server holds the data and logic; clients (web browsers, mobile apps, desktop apps) send requests and render results. This is the most common shape for web and mobile applications, and is often combined with a layered or modular server internally.

Structure

```
client/  
  src/  
    api/          -> functions that call the server  
    components/   -> UI pieces  
    state/        -> local app state  
  server/  
    src/  
      routes/      -> what the client can call  
      controllers/  -> handle requests  
      services/     -> business logic  
      repositories/ -> data access  
  db/
```

Design principles

- **Define the API contract clearly**, ideally with a schema (OpenAPI, GraphQL schema, or similar), so client and server can be developed and versioned somewhat independently.
- **Never trust the client.** All validation and authorization must be enforced on the server; client-side checks are only for user experience.
- **Keep the client thin on business logic.** The client should mostly handle presentation and light state, not reimplement rules that already live on the server.

- **Design for statelessness where possible** on the server side (for example, token-based auth instead of server-side sessions), which makes it far easier to run multiple server instances later.
- **Version the API** so client updates and server updates do not have to ship at exactly the same moment.

Common pitfalls

- Duplicating business rules on both sides and letting them drift out of sync.
- Chatty APIs that force the client to make many small requests to assemble one screen, instead of designing endpoints around what the client actually needs.
- Putting sensitive logic or secrets in client-side code, where anyone can inspect it.

When to use it

The standard choice for essentially any web application, mobile app backend, or multiplayer game with shared state. Almost always the outer shape of a system, with one of the other patterns (layered, modular, monolithic, or microservice) describing what happens inside the server.

Interfaces and design patterns

- **Adapter pattern on the client.** Wrap all server calls behind a small API client module (functions like `getUser`, `createOrder`) so the rest of the client code never constructs HTTP requests directly, and swapping REST for GraphQL later touches one file.
 - **Repository pattern on the server**, same as in a layered architecture, so controllers depend on a data-access interface rather than a specific database.
 - **MVC or MVVM on the client** to separate what's shown (View) from how it's structured (Model) and how user actions are handled (Controller / ViewModel), which keeps UI components from accumulating business logic.
 - **DTO pattern** at the API boundary in both directions: request DTOs validate and shape incoming data, response DTOs control exactly what the client receives, independent of internal server models.
 - **Interceptor / middleware pattern** on the server for cross-cutting concerns (auth checks, logging, rate limiting) applied uniformly to routes instead of repeated in every controller.
 - **Observer pattern on the client** for real-time updates (WebSockets, subscriptions), notifying UI components when server-pushed data arrives.
-

Quick Comparison

Style	Concerned with	Main unit	Best for
Layered	Internal organization	Horizontal layers	Most CRUD apps, clear responsibility split
Modular	Internal organization	Feature modules	Multi-domain apps, future flexibility
Monolithic	Deployment	One process	Small-to-mid teams, simplicity
Microservice	Deployment	Independent services	Large systems, independent scaling/teams
Peer-to-peer	Network topology	Equal nodes	Decentralized, no single point of failure
Client-server	Network topology	Central server + clients	Web and mobile apps (default shape)

Combining styles in practice

- A typical small project: **client-server** shape, **monolithic** deployment, **layered** internal structure.
- A typical growing product: **client-server** shape, **modular monolith** deployment, each module internally layered.
- A typical large-scale product: **client-server** shape at the edge, **microservices** behind an API gateway, each service internally layered or modular.
- A decentralized app: **peer-to-peer** shape, with each node internally structured however suits it, often layered.